Beyond the Agent Object: Entity Component Systems as an Architecture for Agent-Based Modeling

Franz Scharnreitner BSc.

Agent-based models are commonly implemented around agent objects that combine identity, state, and behavior. Although this organization mirrors the intuitive description of autonomous agents, it makes predefined agent types the principal unit of model construction and can obscure the population-level processes through which agents interact. This paper examines Entity Component Systems (ECS) as an alternative architecture for agent-based modeling along three analytically distinct dimensions: the composition of agent state, the organization of behavior, and the semantics of the execution schedule. ECS connects these dimensions through one vocabulary: component signatures define agent structure, queries select the participants in population processes, and system dependencies constrain schedules and update visibility. We compare ECS with an idiomatic agent-centered implementation of the same model to examine how each architecture represents changing roles, organizes shared mechanisms, and declares the conditions under which alternative execution orders preserve outcomes. The argument is developed through a reconstruction and extension of the single-resource Sugarscape wealth-distribution model, including optional reproduction and disease. A secondary engineering evaluation considers cache locality, multicore scaling, and the prospects for GPU execution without treating storage performance as a separate research question. The paper positions ECS not only as a computational optimization, but as an architecture for scientific model specification.

July 25, 2026

Contents

Status of this document

This is a manuscript scaffold. Each section records the claim it should establish, the evidence it requires, and important qualifications. Replace these notes with finished prose only after the corresponding implementation or analysis exists.

1. Introduction

Consider the single-resource Sugarscape wealth-distribution model. Citizens with heterogeneous vision, metabolism, endowment, and lifespan move across a regenerating landscape, harvest sugar, accumulate wealth, and die from starvation or old age [1]. Optional reproduction and disease introduce roles that overlap and change over time: a citizen may be female or male, fertile or infertile, susceptible, infected, or immune, and several of these classifications may apply simultaneously. Some roles require persistent state, while others are derived from age, wealth, and local relationships rather than belonging to a fixed taxonomy of citizen types. This modeling choice exposes an architectural chain: how agent state is composed, how population processes select and transform that state, and how those processes are scheduled. The resulting organization may also affect how simulation data are laid out and executed, but that engineering consequence is analytically secondary to the model specification.

ABMs explain macro-level regularities as the emergent result of heterogeneous agents and their interactions. In most implementations, the scientific agent is mapped onto a software object that combines identity, state, and behavior. This mapping is intuitive, but it is not neutral: it encourages modelers to begin with a taxonomy of agent types, locate behavior in agent-local step functions, and treat population-level processes as consequences of repeatedly invoking those functions.

Entity–component–systems offer a different starting point. Entities supply identity, components represent state and capabilities, and systems implement transformations over every entity possessing a required component signature. The resulting architecture is organized around composition and processes rather than class membership and agent-local methods.

This paper investigates whether that architectural change is useful for scientific ABMs and how it manifests in concrete modeling choices and practices.

1.1. Why architecture matters

In a computational model, an agent may be a person, household, firm, bank, government, or artificial citizen. These entities differ substantively, but an ABM attributes individual state and actions to each of them and lets those states evolve through action and interaction. The software architecture determines how that scientific description is translated into executable state and processes.

This explicit representation makes ABMs particularly well suited to modeling heterogeneous populations. In practice, however, heterogeneity is often encoded either parametrically, through different values of a shared set of attributes, or categorically, through predefined agent types. In the following sections, we distinguish a third form, which we call structural heterogeneity: agents may differ in the state variables and capabilities they possess, independently of any fixed type hierarchy. An entity–component–system architecture supports this form of heterogeneity by constructing agents through the composition of components.

Interactions are rarely the behavior of one agent alone. They are relational processes through which agents jointly produce population-level dynamics and, potentially, nonlinear feedback. Yet many ABM frameworks organize execution around methods invoked on one agent at a time. This can make a process involving several agents appear to belong to a single participant,

obscuring who participates and when the resulting state changes take effect. The distinction becomes particularly important for simultaneous interactions, which require separate phases for observation, intention formation, conflict resolution, and state update.

These semantic issues are closely connected to parallel execution. Specialized frameworks such as FLAME GPU demonstrate that agent-centered models can be executed efficiently in parallel: agents of the same type and state execute agent functions concurrently on the GPU. Safe interaction is achieved through explicit messages, while layers or dependency graphs determine the order in which functions execute. Parallelism therefore remains possible, but requires a comparatively constrained execution model in which agent types, states, messages, and dependencies must be specified explicitly. (FLAME GPU documentation (https://docs.flamegpu.com/guide/creating-a-model/index.html°))

ECS offers a different organizing principle. Even in a structurally heterogeneous population, systems apply homogeneous operations to agents that share particular components. Component-oriented storage can make these operations amenable to batching, while systems expose their data dependencies as a basis for safe scheduling and parallel execution. ECS does not make parallelism automatic, but aligns the unit of execution with population-level processes rather than individual agent methods.

1.2. What Changes with ECS?

We examine ECS as a modeling architecture for ABMs. Three dimensions structure the comparison. They are separated analytically because each supports a different claim, but ECS derives its architectural force from connecting them.

Dimension	Modeling question	ECS commitment
State composition	What state and capabilities constitute an agent?	An entity's component signature defines its current schema
Behavior organization	Where are shared transition laws expressed?	Systems transform populations selected by component queries
Schedule semantics	Which state is visible to each process, when do updates take effect, and which execution orders are equivalent?	System dependencies and phase boundaries specify ordering, visibility, and admissible reorderings

State composition and behavior organization concern the executable description of the model. Schedule semantics determines the transition that description implements. The central thesis is not that these dimensions collapse into one another, but that ECS gives them a common interface: the same component requirements that constitute a role in the scientific model select the population processed by a system, while declared dependencies specify when those transformations may observe and modify state. Component-oriented storage can exploit this information, but its performance is evaluated as a secondary engineering consequence.

The investigation is organized around three questions:

1. **RQ1 — State composition:** How does ECS affect the representation and modification of overlapping and changing agent roles or capabilities relative to an idiomatic agent-centered implementation of the same model?
2. **RQ2 — Behavior organization:** How does organizing behavior as systems affect the locality, reuse, substitutability, and inspectability of population-level mechanisms relative to agent-centered activation?
3. **RQ3 — Schedule semantics:** How do explicit system dependencies and phase boundaries specify information visibility and update timing, and under what declared conditions do alternative execution orders preserve model outcomes?

Our contribution is threefold. Conceptually, we distinguish state composition, behavior organization, and schedule semantics; within the first dimension, we distinguish parametric, type-based, and compositional heterogeneity and map the principal concepts of ABM to entities, components, queries, systems, and resources. Methodologically, we reconstruct the same classical economic ABM in idiomatic agent-centered and ECS architectures, extend it with overlapping and changing behavioral capabilities, and formulate simultaneous interaction as a staged process derived from explicit system dependencies. Empirically, we use controlled architecture and schedule treatments to assess the three questions. A separate engineering evaluation measures cache behavior and multicore scaling while assessing the suitability of component-oriented execution for GPUs.

2. Conceptual Foundations

This chapter defines two concepts used for both architectures: the forms of heterogeneity present in a population and the relation among world states, transitions, schedules, trajectories, and model time. The definitions do not presume where behavior is located or how state is stored.

2.1. Three kinds of heterogeneity

Parametric, type-based, and compositional heterogeneity are distinct but not mutually exclusive dimensions along which agents may differ. A model may combine any or all of them. Let $\mathcal{W}$ be the set of admissible complete world states. For $W \in \mathcal{W}$, let $I(W)$ be the finite, nonempty set of agents present in that state and let $N(W) = |I(W)|$. This formulation permits entry, exit, and replacement; neither population size nor agent identifiers need remain fixed. It also avoids assigning a clock before the model's transition and scheduling semantics have been specified. Sugarscape provides the running example: citizens carry heterogeneous vision and metabolism, occupy sex and infection roles, and enter or leave the population through replacement, reproduction, and death.

A complete W contains the agent descriptors defined below together with the environment and all remaining model state. Where a scheduled transition uses randomness, the current state of the seeded model RNG is included as well; conditional on that complete state and a fixed schedule, the successor is determinate.

For every $i \in I(W)$, let $X_W(i)$ denote the agent's admissible state space in W and let $x_W(i) \in X_W(i)$ be its current state. Thus $X_W : I(W) \to \boldsymbol{X}$ maps agents into a universe $\boldsymbol{X}$ of admissible state spaces. The state space is indexed by the complete world because an agent's schema need not remain fixed and may differ between alternative states at the same model time. It may include dynamic variables such as position, speed, or memory as well as more stable behavioral traits.

To distinguish those roles, for each admissible state space X let $p_X : X \to \Theta_X$ project an agent state onto the scientifically designated parameter or stable-trait coordinates. For any nonempty group $J \subseteq I(W)$ whose members share $X_W(i) = X$, the population exhibits **parametric heterogeneity** within J if $p_X(x_W(i)) \neq p_X(x_W(j))$ for some $i, j \in J$. Differences in ordinary dynamic state, such as position or acquired memory, are state heterogeneity but not parametric heterogeneity under this definition. Sugarscape citizens with the same component signature but different vision or metabolism provide an example of parametric heterogeneity.

A population exhibits **type-based heterogeneity** when a nominal classification $\tau_W : I(W) \to \mathcal{T}$ assigns every agent to one member of a predefined, exhaustive type set $\mathcal{T}$, and $\tau_W(i) \neq \tau_W(j)$ for some $i, j \in I(W)$. The classification induces the pairwise-disjoint partition $I(W) = \cup_{\tau \in \mathcal{T}} I_W^\tau$, where $I_W^\tau = \{i \in I(W) \mid \tau_W(i) = \tau\}$. A type may determine an admissible state space or transition law, but state values and component signatures may still vary within it. For example, Sugarscape's reproductive sexes could be encoded as the nominal types `FemaleCitizen` and `MaleCitizen`. The ECS implementation instead uses exclusive zero-sized tag components; this contrast illustrates that type-based heterogeneity is a property of the

model's nominal classification, not of a particular programming language or an inheritance hierarchy.

A population exhibits **compositional heterogeneity** when agents differ in the sets of components that constitute their modeled structure. Let $\mathcal{U}$ be the universe of agent component types and fix the scientifically relevant component types $c_1, ..., c_m \in \mathcal{U}$. Define the full component-signature function and its structural projection by

$$\kappa_W : I(W) \to \mathcal{P}(\mathcal{U}), \quad \kappa_W^{\mathrm{str}}(i) = \kappa_W(i) \cap \{c_1, ..., c_m\}. \tag{1}$$

Thus $\kappa_W(i)$ is the complete implementation signature, whereas $\kappa_W^{\mathrm{str}}(i)$ is the signature used to analyze compositional heterogeneity. Mandatory components may occur among $c_1, ..., c_m$ but cannot by themselves generate heterogeneity. Transient buffers used only to implement a schedule remain in $\kappa_W(i)$ so systems can query them, but are omitted from that list unless their presence has a substantive interpretation.

At this level, a component is only a tag: membership of c in $\kappa_W(i)$ records that component c is present. This definition assigns neither a data domain nor a parameter block to that tag. It therefore keeps the definition of compositional heterogeneity independent of the ECS storage representation introduced later.

For $k \in \{1, ..., m\}$, define the component-incidence indicator

$$\chi_{ik}(W) = \begin{cases} 1 \text{ if } \ c_k \in \kappa_W^{\mathrm{str}}(i) \\ 0 \text{ otherwise} \end{cases}, \tag{2}$$

and collect the indicators in

$$\chi_i(W) = (\chi_{i1}(W), ..., \chi_{im}(W)) \in \{0, 1\}^m. \tag{3}$$

After fixing any ordering of $I(W)$, the incidence matrix $\boldsymbol{C}(W) = (\chi_{ik}(W)) \in \{0, 1\}^{N(W) \times m}$ describes the population's component composition. For $z \in \{0, 1\}^m$, the empirical distribution of structural signatures is

$$\pi_W(z) = \frac{1}{N(W)} |\{i \in I(W) \mid \chi_i(W) = z\}|. \tag{4}$$

The population is compositionally heterogeneous precisely when $\kappa_W^{\mathrm{str}}(i) \neq \kappa_W^{\mathrm{str}}(j)$ for some $i, j \in I(W)$, or equivalently when $|\{z \in \{0, 1\}^m \mid \pi_W(z) > 0\}| > 1$.

The three architecture-relevant dimensions can be described jointly by associating agent i with

$$a_W(i) = (\tau_W(i), \kappa_W(i), X_W(i), x_W(i)), \quad x_W(i) \in X_W(i). \tag{5}$$

This is not an exhaustive taxonomy of all ways agents may differ. In particular, agents with the same type, component signature, and parameter coordinates may still occupy different dynamic states.

Given two states connected by an admissible model transition $W \to W'$, compositional heterogeneity is **dynamic** if a persistent agent $i \in I(W) \cap I(W')$ satisfies $\kappa_{W'}^{\mathrm{str}}(i) \neq \kappa_W^{\mathrm{str}}(i)$, or

if entry, exit, and replacement change the population distribution so that $\pi_{W'} \neq \pi_W$. Heterogeneity itself is not an ECS innovation. The claim here is that ECS makes component incidence and changes to it explicit without requiring an exhaustive taxonomy of combination-specific agent classes.

2.2. Transitions, schedules, trajectories, and time

The state-indexed definitions above do not assume discrete ticks, synchronous updates, or any other time model. An admissible transition relation $\rightarrow \subseteq \mathcal{W} \times \mathcal{W}$ states which world-state changes the model permits. A schedule Σ selects and composes agent activations or systems and thereby induces a transition operator $T_\Sigma : \mathcal{W} \to \mathcal{W}$ satisfying $W \to T_{\Sigma(W)}$.

A simulation run is a trajectory

$$\omega = (W_0, W_1, ...), \quad W_{n+1} = T_{\Sigma_n}(W_n). \tag{6}$$

Here n is a transition index, not yet scientific or physical time. A model clock is a map $\text{clock} : \mathcal{W} \to \mathbb{T}$ into a totally ordered time domain. Along a trajectory, $t_n = \text{clock}(W_n)$. Only after fixing that trajectory may state-indexed objects be abbreviated by

$$I_n = I(W_n), \quad x_{n(i)} = x_{W_n}(i), \quad \kappa_{n(i)} = \kappa_{W_n}(i), \quad \pi_n = \pi_{W_n}. \tag{7}$$

If each application of a complete schedule advances one discrete tick, then $t = n$ and the familiar notation W_t, I_t, and $x_{t(i)}$ is valid shorthand along that run. Event-based models may instead have irregular t_n, including several transitions at the same clock time.

A complete transition may contain ordered phases. For $p \in \{0, ..., P\}$, write $W_{n,p}$ for the state after phase p, with

$$W_{n,0} = W_n, \quad W_{n,P} = W_{n+1,0} = W_{n+1}. \tag{8}$$

The phase index records semantic visibility and update boundaries, not elapsed model time: several phases may share the same clock value. A further local index, such as a movement micro-step, should be introduced only when the scientific mechanism requires it. This separation lets the same state and transition definitions support sequential, simultaneous, phased, and event-based models.

Simultaneity, concurrency, and parallelism describe different properties of a model and its execution. **Simultaneous model semantics** means that a designated set of decisions is formed from a common information state and that no participant observes another participant's decision as an already committed change. **Concurrent execution** means that computations are independently schedulable and may be interleaved without changing that process. **Parallel execution** occurs when such work is performed at the same time on several hardware execution units. A simultaneous interaction may be evaluated serially, and a sequential model may use parallelism within an individual activation.

Let $K_{n,p}$ be the processes assigned to phase p of transition n, and let P_k produce the proposed update of process k. Under simultaneous phase semantics, the phase-entry state is held fixed while every proposal is formed,

$$\delta_{k,n,p} = P_{k(W_{n,p})}, \quad k \in K_{n,p}, \tag{9}$$

and a joint resolution rule defines the next visible state,

$$W_{n,p+1} = \mathrm{resolve}_{n,p}\left(W_{n,p}, \left(\delta_{k,n,p}\right)_{k \in K_{n,p}}\right). \tag{10}$$

A sequential phase instead evaluates its processes against the intermediate states produced by their predecessors. Many simultaneous interactions can be described by the staged sequence

```
observe -> form intentions -> resolve conflicts -> apply actions -> learn
```

Not every model requires all five stages, and a scientifically sequential interaction should not be made simultaneous only because this decomposition is convenient. The phases specify what each process observes and when its effects become visible; they do not by themselves prescribe a software architecture.

3. Positioning in the Literature

3.1. ABM tools and agent-centered design

The major general-purpose toolkits occupy different positions on the three dimensions introduced above. Their state abstractions nevertheless tend to make an individual agent kind and its fields more explicit than an independently composed component signature. In the notation of Section 2.1, their native abstractions commonly provide τ, X, and x, whereas a scientifically interpreted κ is constructed through classes, interfaces, flags, traits, or subset selectors. Their behavior and schedule abstractions vary more widely: some enter through agent methods, others through external functions, population operations, or explicitly layered kernels [2].

Mesa follows Python's class-based idiom: modelers typically subclass `Agent`, store state in instance attributes, and define behavior as agent methods [3]. A model-level step selects an `AgentSet` and invokes those methods with operations such as `do` or `shuffle_do`, so the population traversal can be filtered, grouped, fixed, or randomized while each invocation still enters through an agent object. Classes and expected attributes provide a natural τ and X, but Python does not impose a closed schema. Mixins, delegated objects, dynamic attributes, capability fields, and `AgentSet` selectors can encode an effective κ and support staged or process-oriented operations over overlapping populations.

MASON makes the nominal route more explicit through Java classes and interfaces. A model commonly defines several agent classes implementing `Steppable`; class fields determine the usual state schema and the scheduler invokes each object's `step(SimState)` method [4]. Helpers, environmental processes, and model-level coordinators may implement the same interface, and its priority queue supports asynchronous events, ordered phases, and fixed or randomized collections. Java interfaces, delegation, composition, and state flags can also represent overlapping capabilities without enumerating every combination as a subclass. Interface incidence may therefore resemble κ more closely than the paper's exclusive τ, although MASON's object and scheduler APIs do not make those capability sets the basis of ECS-style storage queries.

NetLogo separates nominal kinds from classes in the host-language sense. Every individual is first a turtle, patch, or link; `breed` declarations then partition turtles or links into named agentsets, and breed-specific `-own` declarations together with the common `turtles-own`, `patches-own`, and `links-own` declarations specify available variables [5]. A breed is thus a natural τ, the variables available to it define X, and their values form x. Procedures are not methods owned by a class, but `ask` executes them in each selected agent's context and ordinarily applies changes in randomized serial order. Several `ask` passes and temporary state can instead separate decision from application. A turtle or link may also change breed at runtime, making NetLogo an important counterexample to fixed lifetime schemas: changing breed can change the breed-specific part of X_W. Ordinary variables, lists, links, membership predicates, and constructed agentsets can represent overlapping capabilities, but breeds remain exclusive nominal categories rather than independently attachable components.

Agents.jl uses Julia's concrete data types rather than a conventional class hierarchy. A homogeneous model may define one agent struct; heterogeneous models may admit a `Union` of agent types or use `@multiagent` to wrap a closed set of variants [6]. Concrete type or enclosed variant supplies τ, fields determine X, and multiple dispatch can specialize behavior by that

kind. Behavior is defined externally through `agent_step!` and `model_step!`; a scheduler selects the agents and order receiving the former, while model steps, custom schedulers, buffers, and repeated population passes can implement shared or simultaneous processes. `EventQueueABM` additionally supports continuous-time events. Traits, delegated state, flags, and predicates can emulate overlapping κ-like capabilities, but the built-in heterogeneous containers remain organized around a declared union or closed variant set rather than arbitrary component combinations.

FLAME GPU sharpens the distinction because execution efficiency is central to its design. Agent types declare fixed variable schemas, while agent functions are associated with agent states and execution layers and operate over GPU populations [7]. The declared agent type again provides a natural τ and X, but behavior is launched population-wide as GPU kernels rather than invoked serially as object methods. Messages separate communication phases, and ordered layers or a dependency graph determine when agent and host functions execute. Activation is therefore explicitly phased and potentially concurrent, even though eligibility and heterogeneous storage remain organized primarily by agent type and state rather than queries over independently composed capabilities.

The survey yields a sharper baseline than a single agent-centered/ECS binary. These frameworks vary substantially in behavior organization and scheduling, but they primarily anchor heterogeneous state in an agent kind, record, breed, or declared variant. ECS changes that anchor: κ, signature-based selection, and structural change become the native organizing abstractions. The same signature then connects an agent's admissible state to the population processes in which it participates. Storage remains a further implementation question, examined separately below.

3.2. ECS, data-oriented design, and concurrency

- Introduce ECS origins and its emphasis on composition over inheritance.
- Discuss formal work on ECS semantics and deterministic concurrency [8].
- Separate the architectural pattern from particular archetype storage implementations.

3.3. ECS in ABM and multi-agent systems

- Present ECS-based ABM engine work, especially its feasibility and CPU-parallel performance emphasis [9].
- Present HECATE's mapping between ECS and multi-agent engineering [10].
- Review GPU ABM frameworks whose agent functions, execution layers, messages, or kernels already resemble population-level systems.

3.4. Proposed gap

The intended novelty claim is:

"Previous work has established the feasibility and performance potential of ECS-based ABM engines. This paper instead examines ECS as a scientific modeling architecture, focusing on structural heterogeneity through dynamic component composition, the representation of behavior as population-level systems, and the resulting formulation of simultaneous agent interactions."

Literature work still required

Conduct a reproducible search across ABM, individual-based modeling, multi-agent systems, component-based simulation, process-oriented simulation, FLAME/FLAME GPU, and data-oriented scientific computing. Do not use “first” or “previously unexplored” until this search is documented. Add literature on ODD model descriptions, activation regimes, component-based ABM, and GPU conflict resolution.

4. Agent-Centered Baseline

4.1. The agent object convention

A common implementation of an ABM assigns each scientific agent a corresponding software object. The object has an identity and a named type or record structure, while its fields hold both externally visible state, such as position or wealth, and internal state, such as beliefs, preferences, or memory. Behavior is associated with that representation through methods selected by inheritance, dispatch, or explicit calls. We call this organization the **agent object convention**. Here, "object" is used in the broad architectural sense of a state-bearing software representation; it does not require a particular object-oriented language or an inheritance hierarchy.

Execution under this convention is typically organized by a scheduler. At each tick or event, the scheduler selects an agent and invokes a step function or event handler associated with its representation. During that invocation, the agent may inspect its own fields, query nearby agents or shared model state, choose an action, and mutate itself, another agent, or the environment. The population-level transition is consequently assembled from repeated invocations of agent-level behavior. Toolkits differ substantially in how they order these invocations: they may use fixed or randomized activation, simultaneous-update buffers, event queues, multiple agent types, or user-defined schedules [2].

This organization makes the agent object the principal unit of both description and execution. The state belonging to an agent is determined by its record or type, behavior is approached by asking what that agent does during its activation, and an interaction commonly enters the implementation through one participating agent's method. These choices are architectural commitments rather than requirements of agent-based modeling itself. The same scientific model could instead store state separately from identity and express behavior as transformations over every agent satisfying particular conditions.

In this paper, **agent-centered** names the conjunction of two default commitments: the complete agent representation is the primary schema of individual state, and an activation of that representation is the principal entry point to behavior. Schedule semantics and physical storage are treated as separate dimensions rather than inferred from those commitments.

4.2. Behavior located in agents

The preceding forms of heterogeneity describe what may differ between agents; they do not yet specify where the corresponding behavior is implemented. With the notation of Section 2.1, $a_{W(i)}$ is the descriptor of agent i in world state W. Let $N_i(W)$ be the information about that state made available to agent i. In an agent-centered architecture, activating $i \in I(W)$ invokes an agent-level transition operator

$$F_{i(W)} = f_{\tau_{W(i)}}\big(\kappa_{W(i)}, x_{W(i)}, N_{i(W)}, W\big). \tag{11}$$

The right-hand side is understood to return an updated world state. The type $\tau_{W(i)}$ may select the implementation by dispatch, $\kappa_{W(i)}$ may select branches or delegated behaviors associated with available components, and $x_{W(i)}$ supplies the agent-level state used by those behaviors,

including the parameter coordinates $p_{X_{W(i)}}\left(x_{W(i)}\right)$. Any additional model state is already part of W. The operator may update the agent, other agents, or the environment; it may also replace $\kappa_{W(i)}$, $X_{W(i)}$, and $x_{W(i)}$ with a new signature, corresponding state space, and admissible state value. Agent-centered organization therefore does not preclude compositional or dynamically changing heterogeneity. Its defining feature is that these transformations are reached through the activation of a particular agent representation.

A scheduler turns the individual operators into a population transition. For a current state W, let $\sigma_W : \{1, ..., N(W)\} \to I(W)$ give the activation order of the agents scheduled from that state. In the simplest sequential case,

$$W^0 = W, \quad W^r = F_{\sigma_W(r)}(W^{r-1}), \quad T_{\sigma_W}(W) = W^{N(W)}. \tag{12}$$

Agent $\sigma_W(r)$ consequently observes $N_{\sigma_W(r)}(W^{r-1})$: an earlier activation may change the state seen by a later one. Activation order is behaviorally relevant whenever two operators do not commute,

$$F_i \bigcirc F_j \neq F_j \bigcirc F_i. \tag{13}$$

This formulation also shows why a single agent step can obscure timing. One invocation may successively perform perception, choice, action, interaction resolution, and learning even though the scientific model assigns those processes different information sets or update times. Reading and writing the world during the same invocation gives each process the intermediate state created by the preceding code unless the implementation introduces explicit buffers or phases.

Behavior organization and schedule semantics become distinct as soon as an agent activation produces an intention rather than immediately mutating the world. A simultaneous schedule can first evaluate

$$p_i = G_i(W), \tag{14}$$

and then resolve and apply the collection $(p_i)_{i \in I(W)}$ in a model-level operation. The Sugarscape comparison uses the same citizen state in two such schedules: its shuffled-sequential treatment moves and harvests for one citizen before activating the next, whereas its synchronous treatment forms all destination proposals from a frozen occupancy and landscape state before resolving contested cells. This isolates the schedule's information semantics from the location of the behavior that produces each intention.

4.3. From agent types to state schemas

Many traditional ABM frameworks make the agent's declared kind the default place in which its admissible fields are specified. In the notation of Section 2.1, this common design can be represented by a schema map $\tilde{X} : \mathcal{T} \to \boldsymbol{X}$ such that $X_W(i) = \tilde{X}(\tau_W(i))$. The map need not be surjective: several types may share a schema, and some admissible spaces in $\boldsymbol{X}$ may be unused. Nor is this relationship necessary. Dynamic fields, delegated objects, flags, traits, and model-level tables can make the effective state schema depend on more than the nominal type.

Where both the software type and its declared schema remain fixed over an agent's lifetime, one may write $\tau_W(i) = \tau(i)$ and $X_W(i) = X(i)$ for every admissible W containing i. This is a frequent framework default, not a universal property of agent-centered modeling.

4.4. What this architecture does well

- It closely follows ordinary-language descriptions of autonomous individuals.
- The complete state and behavior of one agent can be inspected in one place.
- Highly individualized cognition or event-driven behavior can be natural to express.
- Mature ABM frameworks provide extensive tooling around this representation.

4.5. Limitations to investigate rather than assume

- Growth of type hierarchies or conditional behavior as capabilities overlap.
- Duplication when several agent types share a process.
- Difficulty separating observation from mutation when simultaneous behavior is required.

Required standard of comparison

Use an idiomatic agent-centered implementation, not a deliberately weak inheritance hierarchy. Julia is not a conventional class-based OOP language, so describe the Agents.jl implementation as agent-centered. If the paper makes stronger claims about OOP, add a representative class-based implementation or narrow the terminology.

5. Entity Component Systems as a Modeling Architecture

Building on the engine and multi-agent work reviewed in Section 3, this chapter turns from ECS feasibility to its consequences for scientific model construction.

The ECS pattern first changes two aspects of model specification: how agent state is composed and where transition laws are located. Queries and declared effects then connect those choices to a third dimension, the schedule that orders processes and defines update visibility. An engine may exploit the same information for component-oriented storage and batch execution, but storage does not define the scientific semantics, and a schedule is not implied by a memory layout. The architectural claim is more specific: ECS carries one explicit account of required state from agent composition, through process participation, to dependency and schedule analysis. Its consequences for the loops that execute the model are evaluated separately.

5.1. Entities, components, systems, and resources

- **Entity:** an identifier with no intrinsic data or behavior.
- **Component:** a focused unit of state associated with an entity.
- **Component signature:** the set of components currently associated with an entity.
- **Query:** a state-dependent selection of entities by component signature and optional predicates.
- **System:** a transformation over components selected by one or more queries.
- **Resource:** model-level state such as parameters, random-number streams, spatial indexes, or aggregate statistics.
- **Structural change:** the addition or removal of entities or components.

Parallel to Section 4.3, ECS binds an entity's state schema to its component signature rather than necessarily to one nominal type. Formally, let $\Phi : \mathcal{U} \to \mathbf{Set}$ assign a state-value domain to each component type. The admissible state space of entity i is then

$$X_W(i) = \prod_{c \in \kappa_W(i)} \Phi(c). \tag{15}$$

When the component domains separate dynamic state from stable traits, this can equivalently be written $X_W(i) \equiv S_W(i) \times \Theta_W(i)$. More generally, the projection $p_{X_W(i)}$ identifies the coordinates treated as parameters for the scientific comparison. A parameter that mutates or changes at replacement is therefore part of the complete simulation state even while serving as a parameter of the agent's behavioral transition law.

5.1.1. Queries

A query describes a role that entities may occupy in a model process. It is evaluated against a world state rather than identified once and for all with a fixed subset of agents. For $W \in \mathcal{W}$, use the entity set $I(W)$ and component-signature function κ_W defined in Section 2.1.

Represent a query Q by required and excluded component sets $\mathcal{C}_Q^+, \mathcal{C}_Q^- \subseteq \mathcal{U}$ and an optional state predicate ψ_Q. Its result is

$$E_Q(W) = \left\{ i \in I(W) \mid \mathcal{C}_Q^+ \subseteq \kappa_W(i), \mathcal{C}_Q^- \cap \kappa_W(i) = \varnothing, \psi_Q(i, W) = 1 \right\}. \tag{16}$$

Taking $\mathcal{C}_Q^- = \varnothing$ and $\psi_Q = 1$ gives an ordinary required-signature query. Value predicates may select a narrower current population, while an excluded-component condition can distinguish, for example, entities that have position but lack a movement capability. Query membership may therefore change when component composition or component values change, even if the entity persists.

A system may use a finite query family

$$\boldsymbol{Q}_k = \big(Q_{k1}, \ldots, Q_{km_k}\big), \tag{17}$$

with $m_k \geq 0$. The corresponding population vector at W is

$$\boldsymbol{E}_k(W) = \big(E_{k1}(W), \ldots, E_{km_k}(W)\big), \quad E_{kj}(W) = E_{Q_{kj}}(W). \tag{18}$$

The empty family permits a process that operates only on model-level state. Multiple queries may identify different participant roles in one interaction, or they may provide complete populations that a system matches, aggregates, or otherwise processes jointly. This is broader than treating every query result as an independent invocation, while retaining the Core ECS idea that systems declare their inputs through queries [8].

5.1.2. Resources

A resource is state belonging to the model as a whole rather than to one entity. Let $\mathcal{G}$ be the finite set of resource labels and let $\Psi : \mathcal{G} \to \mathbf{Set}$ assign a value domain to each resource. The resource part of world state W is

$$r_W \in \prod_{g \in \mathcal{G}} \Psi(g). \tag{19}$$

Parameters, the seeded random-number generator, occupancy indexes, environmental trace fields, replacement queues, and aggregate statistics are resources in this sense. A resource may be read-only during a run or may evolve as part of the simulation state. In particular, stochastic behavior remains a state transition conditional on the current state of the model RNG resource.

For dependency declarations, take $\mathcal{A} = \mathcal{U} \cup \mathcal{G}$ as a tagged union of component and resource labels. A system's declared read and write sets are subsets of $\mathcal{A}$. They are conservative, type-level summaries: an invocation may touch only particular entities carrying a component, but the declaration records every component or resource kind that it may access.

5.1.3. Systems

A system specification is given by the tuple

$$\mathcal{S}_k = (\boldsymbol{Q}_k, \mathrm{Read}_k, \mathrm{Write}_k, F_k), \tag{20}$$

where $\boldsymbol{Q}_k$ is its query family, $\mathrm{Read}_k, \mathrm{Write}_k \subseteq \mathcal{A}$ declare its possible effects, and F_k is its system function. Let $\rho_k(W)$ denote the projection of W containing the component and resource values declared in Read_k. From the selected entities and those inputs, F_k determines

the new values of the declared outputs. Evaluation of the system therefore induces a state transition

$$T_k : \mathcal{W} \to \mathcal{W}. \tag{21}$$

Values outside Write_k remain unchanged under T_k.

Every component or resource inspected by a query, including a component tested only for presence or absence, is included in Read_k. Adding or removing such a component is correspondingly a write to that component label. The declarations therefore cover changes to query membership as well as reads and writes of component values.

The function may update component or resource values and may create or remove entities or components. A schedule determines when queries observe state, when updates become visible, and how several systems compose, as formalized in Section 2.2. An implementation may defer structural changes until it can safely update its storage, but this is an engine constraint rather than part of the definition of a system.

When the queries describe distinct participant roles, the system may define a match relation

$$\mathcal{M}_k(W) \subseteq \prod_{j=1}^{m_k} E_{kj}(W). \tag{22}$$

The relation may exclude self-interaction, impose spatial or institutional eligibility, or otherwise avoid treating the complete Cartesian product as a set of interactions. A tuple-wise system applies a kernel f_k to the matches. For each $\boldsymbol{i} \in \mathcal{M}_k(W)$, define the indexed proposal

$$d_{k,\boldsymbol{i}}(W) = f_k(\boldsymbol{i}, \rho_k(W)). \tag{23}$$

Under simultaneous semantics all kernels observe the same W. Writing $\boldsymbol{d}_k(W)$ for the family of these indexed proposals, joint resolution defines the system transition

$$T_k(W) = \mathrm{resolve}_k(W, \boldsymbol{d}_k(W)). \tag{24}$$

A collection-wise system instead gives F_k the complete population vector $\boldsymbol{E}_k(W)$ and lets it perform matching, aggregation, sampling, or arbitration internally. Market clearing, synchronous path-conflict resolution, and replacement from a survivor pool are naturally expressed this way. If different queries within what appears to be one process must observe different intermediate states, that timing is part of an internal schedule; where scientifically meaningful, the process should instead be decomposed into separate ordered systems.

This representation connects model semantics, dependency analysis, testing, and possible parallel execution. Systems whose writes cannot affect another's reads or writes may admit concurrent evaluation. Contending writes or a write that changes another system's inputs require ordering, reduction, arbitration, or a separate update phase.

5.2. Compositional heterogeneity

The query mechanism operationalizes the structural definition in Section 2.1. An entity participates in a process because its current signature satisfies that process's query. Independently

evaluated queries may select overlapping populations: a firm may be both an exporter and a borrower, or a Sugarscape citizen may be female and infected while also satisfying the age and wealth predicates for fertility, without requiring a combination-specific citizen type.

The incidence vector $\chi_{i(W)}$ therefore describes both an agent's state schema and its eligibility for processes. A newly occurring signature does not by itself require a new implementation because each system continues to select only the components it needs. If a mechanism depends specifically on a conjunction of capabilities, however, that conjunction must still be expressed in a query or behavioral rule.

Structural changes alter query membership as well as state schema. Their timing remains a schedule choice: an engine may stage component additions and removals until a phase boundary so a query is not invalidated during traversal. ECS makes that change explicit but does not decide when it should become scientifically visible.

5.2.1. Example component-incidence matrix

Entity	Position	Female	Male	Infection
Citizen A	✓	✓		✓
Citizen B	✓	✓		
Citizen C	✓		✓	✓
Citizen D	✓		✓	

The mandatory `Position` column is identical for all four citizens and hence does not contribute to heterogeneity in this population. `Female` and `Male` are mutually exclusive tags, while `Infection` overlaps either sex. Infection progression selects Citizens A and C; reproduction first selects the appropriate sex tag and then evaluates age, wealth, and neighborhood predicates. Recovery removes `Infection` from A or C without changing identity or unrelated state, so query membership and component composition change within a citizen's life. The example also shows that not every changing role must be a component: fertility is derived from existing state rather than stored as a tag.

5.3. Thinking in systems rather than agents

The agent-centered question is, "What does this agent do during its step?" The system-centered question is, "What transformation occurs in the model, and which entities possess the state required to participate?" This change of question shifts the primary unit of executable organization from the complete agent representation to a population-level process.

First, this organization makes processes explicit. Perception, choice, movement, matching, learning, entry, and exit can be represented as distinct transformations, each with its own participants and visibility boundary. The model description need not recover a population process by tracing the order in which fragments of it are invoked from individual agent steps. Conversely, the decomposition forces the modeler to decide whether two operations are one process or ordered processes and whether the output of one is visible to the other. A one-to-one correspondence between a scientific process and a software system is not automatic: a complex market-clearing mechanism may require several systems, while a simple mainte-

nance system may combine several scientifically unimportant updates. The benefit is that this correspondence becomes an explicit modeling choice.

Sugarscape illustrates the resulting organization. Growback transforms the landscape; movement selects positioned citizens with vision and wealth; synchronous resolution operates on all submitted destinations; disease progression selects only infected citizens; lifecycle rules process metabolism and age; and reproduction matches eligible female and male populations. A citizen participates in several of these processes during one period, but no single invocation of that citizen is responsible for advancing the period. The executable structure instead follows the sequence of processes that jointly produce citizen and landscape transitions.

Second, systems provide boundaries at which mechanisms can be substituted. An alternative movement, transmission, inheritance, or matching rule can replace F_k while retaining the entities, component storage, and unrelated systems, provided it honors the same query and state contract. If the alternative requires new inputs or produces different state, the affected declarations and downstream dependencies must change as well; system separation does not make scientifically incompatible mechanisms interchangeable. It does, however, localize the change and permits competing mechanisms to be tested against the same initialization, surrounding processes, and recorded outcomes. Mechanism substitution thereby becomes a controlled model comparison rather than a new taxonomy of agent variants.

Third, the declared dependencies make coupling between processes inspectable. A read of another system's output establishes a possible ordering requirement; overlapping writes identify a need for sequencing, reduction, or arbitration; and disjoint outputs with read-only shared inputs identify candidates for independent evaluation. These declarations also supply useful testing boundaries: a system can be tested on a constructed world state, its permitted outputs can be checked, and invariants can be examined at phase boundaries. Read and write sets are conservative access summaries, however, not a proof of scientific correctness. They reveal which state a system may use or alter, but not whether its behavioral equation, information set, or placement in the schedule is substantively justified.

Locating behavior in systems does not remove agency from the scientific model. An agent remains an identifiable unit to which the model attributes private information, goals, dispositions, memory, feasible actions, and consequences. Those entity-specific quantities reside in components; a system implements the transition law shared by entities that currently possess the state required by that law. Autonomy is therefore a property of the modeled information and decision structure, not of whether executable behavior is stored in a method attached to an object. Highly individualized cognition may still be represented through entity-specific policy state or specialized components, although in such models a system-oriented decomposition can make the complete behavior of one agent harder to inspect in one place.

The architectural gain is the alignment between state composition and process organization. A component has a dual role: it contributes to the state of an individual entity and makes that entity eligible for systems requiring it. System read and write sets extend the same contract into the schedule. The modeler can therefore move from "which agents possess this capability?" to "which process uses it, what state does that process observe, and when do its

effects become visible?" without translating between a type hierarchy, an activation routine, and a separate dependency description.

5.4. Interaction phases and concurrency

The phase semantics defined in Section 2.2 become executable in ECS through queries, proposal state, system read and write sets, and explicit resolution systems. A system proposal need not correspond to an independently elapsed instant or an immediately visible change: its temporal meaning follows from the phase in which its query is evaluated and the boundary at which its effects are committed.

The common snapshot also separates proposal production from conflict resolution. Proposal functions can be evaluated concurrently when they treat $W_{n,p}$ as immutable and write to isolated proposal state. Their eventual effects may still conflict; `resolve` must then define which proposals succeed, how joint quantities are aggregated, and which changes enter $W_{n,p+1}$. A stable resolution rule is part of the model, not merely a synchronization detail. In synchronous Sugarscape movement, for example, all destination proposals are fixed before contenders for the same cell are resolved, and no citizen wins merely because its proposal was computed first.

Read and write declarations give a conservative condition for systems that can be applied independently without a separate joint resolver. For distinct systems j and k, the absence of the conflicts

$$\mathrm{Write}_j \cap (\mathrm{Read}_k \cup \mathrm{Write}_k) = \varnothing, \quad \mathrm{Write}_k \cap \big(\mathrm{Read}_j \cup \mathrm{Write}_j\big) = \varnothing \tag{25}$$

means that neither can change an input or output of the other. Subject to the declared accesses being complete, their transitions commute and their evaluation order cannot affect the successor state. Shared read-only inputs do not create a conflict. Because the declarations operate at the component or resource level, this test is sufficient but not necessary: two systems may write the same component type on provably disjoint entity sets, and commutative reductions may permit controlled overlapping contributions. Such cases require a more precise partition, reduction rule, or arbitration step rather than an uncoordinated shared write.

Determinism additionally requires the proposal and resolution functions to be independent of runtime scheduling. A shared mutable random-number generator is a write dependency: allowing concurrent systems to draw from it in an unspecified order can assign different shocks to different entities. Random draws must therefore be deterministically partitioned or indexed by replicate, phase, entity, and event when results are intended to remain invariant to thread scheduling. Reductions require similar care, because an unspecified accumulation order can change floating-point results even when the mathematical operator is associative. Fixed partitions, stable tie-breaking, and explicit reduction orders make these choices reproducible. Formal accounts of Core ECS likewise identify classes of programs whose outcomes are independent of system scheduling, establishing deterministic concurrency as a property that can be derived from restrictions on system effects rather than assumed from the ECS label alone [8].

ECS therefore exposes opportunities for concurrency but does not eliminate interaction dependencies. Systems that contend over shared resources or entity state still require order-

ing, buffering, reduction, or arbitration, and phase barriers introduce synchronization costs. Whether concurrent evaluation produces a speedup depends on the amount of independent work relative to those costs. More importantly, the dependency graph can preserve only semantics that have first been declared: ECS cannot decide whether agents should act simultaneously, which conflicts should be resolved together, or when learning should observe an outcome. Those remain scientific commitments of the model.

5.5. Data-oriented engineering consequences

Component composition is a logical description of model state; component storage is its physical realization. The distinction matters because an ECS could store each entity as a map from component type to value, while a record-based agent population could be transformed internally into a structure-of-arrays layout. Neither representation alone establishes a performance result. The engineering opportunity arises when the logical component signature is also used to organize storage and execution.

In an array-of-structures layout, consecutive memory locations contain complete agent records. A population sweep that uses only position and speed still traverses records containing every other field and may load cache lines filled partly with unused state. A component-oriented layout stores values of the same component together, or groups entities with the same signature into archetype tables whose columns are component arrays. A system query can then produce batches in which the required columns are contiguous and the same kernel applies to every selected row.

This is where the three modeling dimensions can inform execution. The component requirements that give a citizen a behavioral role also select the tables traversed by the corresponding system. Its read and write declarations identify the columns that must be available and the conflicts relevant to concurrent scheduling. The engine need not rediscover those populations and dependencies from branches inside a complete agent step. Homogeneous inner loops can consequently load less irrelevant state, admit vectorization, and be partitioned across CPU threads. Where component arrays are regular and interactions can be expressed through bounded messages or staged kernels, the same batches can be transferred to GPU execution.

These gains depend on workload. Small populations may not amortize query and dispatch overhead. Adding or removing a component can move an entity between archetypes and copy its state. Sparse signatures can fragment batches, while branch-heavy cognition and tightly coupled interactions reduce vector and GPU utilization. Phase barriers, reductions, and conflict resolution can dominate the component sweeps they coordinate. Component-oriented storage can also make the complete state of one individual less convenient to inspect even when it makes population operations clearer.

Storage performance must therefore be evaluated independently of the three research questions. A valid benchmark holds the modeled transition fixed while varying layout and execution policy; it then separates serial data-layout effects from thread scaling and from the cost of queries, structural changes, synchronization, and resolution. The secondary engineering evaluation asks when the alignment offered by ECS pays for these costs, not whether the ECS label alone guarantees faster execution.

6. Comparative Case Study: Sugarscape

6.1. Why Sugarscape

Sugarscape is a canonical agent-based model in which heterogeneous citizens move across a spatial resource landscape, harvest sugar, consume it through metabolism, accumulate unequal wealth, and die from starvation or old age [1]. The repository implements the single-resource wealth-distribution model on a toroidal, two-hill landscape and adds optional sexual reproduction and disease.

The model is especially suitable for the three research questions. Vision, metabolism, initial endowment, and maximum age provide parametric heterogeneity. `Female` and `Male` tags provide mutually exclusive nominal roles. Infection is a genuinely dynamic structural role: transmission adds an `Infection` component, recovery removes it, and the same citizen may remain in the world throughout that change. Reproductive eligibility is instead derived from sex, age, wealth, and neighborhood state. Sugarscape therefore lets the comparison distinguish a role represented by component presence from a temporary role selected by predicates over ordinary state.

Its mechanisms also operate at different population scales. Movement combines individual choice with competition for cells and resources; disease couples neighboring citizens; lifecycle rules remove entities; reproduction matches eligible parents and creates offspring; and resource growback transforms the environment. Finally, the implementation provides both shuffled-sequential and staged synchronous movement. It can therefore separate behavior organization from schedule semantics while producing substantively interpretable outcomes such as wealth inequality, mortality, population change, and infection prevalence.

6.2. Scientific model and extension boundary

The baseline landscape is a finite toroidal grid. Each patch has a fixed sugar capacity and a current stock that grows toward that capacity by a constant amount each period. A citizen observes cells in the four cardinal directions up to its vision, excludes cells occupied by another citizen, and chooses among the cells with the greatest current sugar. Distance breaks sugar ties and the seeded model RNG resolves any remaining tie. After moving, the citizen harvests all sugar at its destination. Metabolism and ageing then reduce wealth and remaining lifetime; starvation or old age removes the citizen.

The default wealth-distribution treatment replaces every death with a newly initialized citizen and disables reproduction and initial infection. The heterogeneous extension instead permits reproduction and disease. Replacement and reproduction are mutually exclusive regeneration regimes. Reproduction matches an eligible female with an adjacent eligible male, reserves an empty neighboring cell, transfers parental endowment to one child, and independently inherits vision, metabolism, and maximum age from either parent. Disease uses one active `UInt64` strain per infected citizen. Cardinal contact can transmit the strain to a nonimmune neighbor, infection imposes a sugar cost, and recovery stores exact-strain immunity before removing the `Infection` component. This is a bounded extension for architectural comparison, not a reproduction of every mechanism in the original Sugarscape.

6.3. Semantics-matched agent-centered reference

The repository currently contains the ECS implementation. A valid comparative study must add an idiomatic agent-centered reference of the same scientific model before drawing conclusions for RQ1 or RQ2. That reference should use one mutable citizen record containing identity, position, proposal, traits, wealth, age, sex, immunity, and an optional infection state. Model-level landscape, occupancy, event counters, clock, and seeded RNG should remain explicit shared state. Movement, transmission, lifecycle, and reproduction may be implemented as ordinary functions reached from citizen activation and model-level coordination; the reference should not be weakened through an artificial class hierarchy.

The two implementations must share parameters, initial landscape and population, indexed random draws, decision and conflict rules, phase semantics, and logger schema. Deterministic fixtures should compare complete world states after every phase. Only state representation and the route by which shared transition laws are reached may differ in the architecture treatments. This constraint prevents the current ECS implementation from being compared with an agent-centered model that implements different movement, disease, or reproduction semantics.

6.4. From agent records to ECS

6.4.1. State composition

In the ECS implementation, a citizen is an entity with the core components `CitizenId`, `Position`, `ProposedPosition`, `Vision`, `Metabolism`, `Sugar`, `Age`, `MaximumAge`, `InitialEndowment`, and `ImmuneProfile`. Exactly one of the zero-sized `Female` and `Male` tags records sex. `Infection` is optional and contains the active strain and infection age. Landscape sugar, occupancy, the seeded RNG, clock, next identifier, step events, parameters, and logger are resources because they describe the world or the run rather than one citizen.

The shift is not merely a decomposition of record fields into smaller records. Component presence participates in the model definition. Adding `Infection` moves a citizen into the population processed by infection progression; removing it on recovery removes that citizen from the query while retaining identity and all unrelated state. A female citizen may simultaneously be infected, fertile, wealthy, or close to death without requiring a combination-specific citizen type. Sex and infection are represented structurally, whereas fertility and mortality risk are computed from component values. This mixed representation makes explicit which roles require stored state and which are transient classifications.

In the agent-centered reference, the same transition can be represented by changing an optional field within the citizen record. The comparison should therefore ask where the schema change, validation, initialization, mutation, logging, and downstream selection logic reside in each architecture. It should not treat dynamic composition as impossible in an agent record.

6.4.2. Behavior organization

Sugarscape's period is decomposed into systems with focused population and resource contracts:

1. `growback!` updates the landscape independently of citizen state.

2. movement either activates shuffled citizens sequentially or separates proposal from conflict resolution and commitment.
3. `disease!` snapshots infectious neighbors, stages new infections, charges the disease cost, and stages recovery.
4. `lifecycle!` metabolizes and ages citizens, stages deaths, removes them, and optionally creates replacements.
5. `reproduce!` first plans compatible births and reserves cells, then deducts parental contributions and creates offspring.
6. `logger!` records population, wealth, resource, movement, mortality, reproduction, and disease outcomes.

This organization makes population mechanisms directly inspectable: their queries identify participants, their resource accesses identify shared inputs, and structural changes are committed after query traversal. The same decomposition creates substitution boundaries. Sequential and synchronous movement can share destination selection; alternative transmission or inheritance rules can retain unrelated lifecycle and landscape systems when they honor the same state contract. The agent-centered reference may achieve the same modularity through functions and helper objects. RQ2 concerns how naturally each architecture exposes and reuses these boundaries, not whether only ECS can modularize behavior.

6.4.3. Schedule semantics

One ECS period has the declared order

```
grow back sugar
  -> rebuild occupancy
  -> move and harvest
  -> transmit and progress disease
  -> metabolize and age
  -> remove deaths and optionally replace them
  -> plan and commit births
  -> advance the clock and log
```

Several dependencies are scientific rather than incidental. Growback precedes choice because citizens observe the replenished landscape. Movement precedes transmission because contact uses post-movement positions. Disease precedes lifecycle because its sugar cost can cause starvation in the same period. Removal precedes reproduction because death changes parent eligibility and available cells. Logging follows all commits so it describes the successor state.

Movement supplies the main schedule treatment. Under shuffled-sequential movement, the model RNG orders citizens; each citizen immediately vacates its origin, selects among currently available cells, moves, and harvests. Later citizens therefore observe earlier movements and depleted patches. Under synchronous movement, every proposal is calculated from the same occupancy and landscape state. A citizen cannot target a cell occupied at the beginning of the movement phase, even if its occupant proposes leaving. When several citizens propose the same initially empty cell, seeded random arbitration selects one winner; losers remain at their origins. Only after resolution are positions, wealth, landscape stocks, and occupancy committed.

Alternative execution orders preserve outcomes only under declared conditions. Within a phase, systems or entity kernels may be reordered when their complete read/write sets are conflict-free, query membership is stable until the phase boundary, resolution is deterministic, and random draws and reductions do not depend on runtime order. The current model uses one shared RNG, so arbitrary system reordering is not outcome-preserving unless draws are first partitioned or indexed by phase, entity, and event. Reordering growback after movement, disease after lifecycle, or reproduction before removal changes information visibility and implements a different model rather than an optimization.

6.5. Comparative treatments and evidence

The main comparison should use a common set of deterministic fixtures and paired-seed experiments.

1. **RQ1:** Compare the record field and optional-state representation with component tags and dynamic `Infection` attachment. Trace infection, recovery, birth, death, and the addition of one further optional capability through state definition, initialization, modification, selection, and logging.
2. **RQ2:** Compare the locality, reuse, substitutability, and inspectability of movement, transmission, lifecycle, and reproduction. Include both the view of one population mechanism and the countervailing task of reconstructing one citizen's complete behavior.
3. **RQ3:** Contrast shuffled-sequential and synchronous movement to measure the consequences of visibility and commitment timing. Separately permute only dependency-admissible systems or kernels and verify complete trajectory equality under indexed randomness and deterministic reductions.

The primary scientific outcomes are population size, mean and median wealth, the Gini coefficient, total citizen and landscape sugar, movement and harvest counts, contested destinations, deaths by cause, replacements, births, infection prevalence, new infections, and recoveries. Architecture comparisons must first establish state and outcome equivalence for the semantics-matched implementations. Schedule treatments intentionally need not be equivalent; they should report how timing changes inequality, resource access, survival, population dynamics, and disease spread.

7. Evaluation Design

7.1. Evaluation logic

The paper cannot establish an architectural advantage from one attractive code example. Each research question therefore receives a distinct treatment and evidentiary standard.

RQ	Treatment	Evidence	Inference
RQ1	Semantics-matched implementations; sex, fertility, and infection roles; infection attachment and recovery	State representation and modification paths across the same model transition	Effects on representing and changing roles, not an inability of agent objects to compose
RQ2	Organize the same mechanisms around systems or agent activation; substitute movement, transmission, reproduction, or resolution rules	Change locality, reused processes, shared contracts, boundary tests, and mechanism traces	Effects on locality, reuse, substitutability, and population-level inspectability
RQ3	Declared phases; admissible reorderings; deliberately changed visibility and update timing	Trajectory equivalence, behavioral outcomes, and counterexamples when declared conditions fail	How schedules specify semantics and the conditions under which order preserves outcomes

7.2. Experiment A: State composition

- Implement the same state variables, mechanisms, schedule, parameters, and seeded random draws in idiomatic agent-centered and ECS forms.
- Construct matched populations with overlapping sex, fertility, and infection roles and exercise infection attachment and removal at declared phase boundaries.
- Add one new capability after both implementations are complete.
- Compare how each architecture represents capability presence and performs a role change, including state definitions, validation, initialization, transition logic, logging, and analysis.
- Report qualitative dependency changes and limited quantitative measures such as touched modules, duplicated transition logic, and combinations represented.

The conclusion should concern locality and composability, not the impossibility of implementing the same model with objects.

7.3. Experiment B: Behavior organization and system substitution

- Hold the scientific state, schedule, entities, and unrelated mechanisms constant across the semantics-matched implementations.
- Compare **locality** by tracing the code and declared dependencies that must change for one population-level mechanism.

- Compare **reuse** by identifying unchanged mechanism implementations shared across capability profiles and experimental variants.
- Compare **substitutability** by exchanging movement, transmission, reproduction, or conflict-resolution rules under the same input/output contract and testing the contract boundary.
- Compare **inspectability** by asking whether a reader can recover a mechanism's participants, inputs, outputs, and schedule position from its implementation; separately report the ease of reconstructing one agent's complete behavior.
- Use movement-mode, reproduction, and disease treatments to illustrate scientific experiments enabled by mechanism separation.

7.4. Experiment C: Schedule semantics

Use two distinct schedule comparisons. First, compare shuffled-sequential and synchronous movement; this identifies the consequences of changing information visibility, resource depletion, conflict resolution, and commitment timing. Second, hold the declared phase semantics fixed and execute multiple admissible topological orders of systems within each phase. For every proposed reordering, state the condition that is expected to preserve outcomes: complete read/write declarations, conflict-free effects or an explicit commutative reduction, stable query membership, schedule-independent random draws, deterministic conflict resolution, and a fixed reduction order where floating-point exactness is claimed. Compare complete trajectories, not only aggregate summaries, and add counterexamples in which a dependency or phase boundary is deliberately violated.

For schedule treatments that intentionally change semantics, hold behavioral equations and initial conditions constant and measure:

- Mean and median wealth and the Gini coefficient.
- Total citizen and landscape sugar and harvested sugar.
- Movements, contested destinations, and deaths by cause.
- Population, births, replacements, infection prevalence, and recoveries.
- Sensitivity to density, resource regeneration, reproduction, and disease parameters.

This experiment distinguishes two claims: different visibility or update timing may change model outcomes, whereas alternative orders satisfying the declared independence conditions should preserve them.

7.5. Secondary engineering evaluation: Storage and execution

Benchmark at least:

1. Agent-centered serial implementation.
2. ECS serial implementation.
3. ECS threaded implementation.
4. ECS GPU implementation, only if it is complete enough for a fair comparison.

Report:

- Wall-clock time after compilation and warm-up.
- Population and component-count scaling.

- Allocations and peak memory.
- Cache misses and memory bandwidth where measurable.
- Thread scaling and parallel efficiency.
- Time spent in queries, systems, structural changes, synchronization, and conflict resolution.
- Hardware, software versions, compiler settings, and number of repetitions.

Use both compute-light and interaction-heavy workloads. A synthetic component-sweep benchmark can isolate memory layout, but Sugarscape must show end-to-end behavior under baseline, reproduction, and disease treatments.

7.6. Randomness, inference, and reproducibility

- Pre-generate or index stochastic draws by replicate, time, entity, and event so architectural treatments do not merely consume different RNG streams.
- Distinguish matched initialization from genuinely paired subsequent shocks.
- Predefine primary contrasts and outcome measures.
- Report Monte Carlo uncertainty and effect sizes.
- Use longer horizons and multiple initial conditions.
- Archive code, project manifests, raw replicate data, and exact reproduction commands.

8. Results

Do not turn this section into prose yet

Organize results by research question, not by the order in which scripts were written. Each subsection should begin with a one-sentence answer, then show the evidence and its uncertainty.

8.1. RQ1: State composition

- Report the number and distribution of component signatures used in the experiment.
- Show how the same overlapping roles are represented and how a capability is introduced or removed in both architectures.
- Report code-locality or dependency evidence without treating code size as scientific proof.
- Separate architectural effects from scientific effects of changing the capability distribution.

8.2. RQ2: Behavior organization

- Present the system dependency graph.
- Show which systems are reused across behavioral variants.
- Report unit and integration tests at system boundaries.
- Compare locality, reuse, and substitutability against the semantics-matched agent-centered implementation.
- Discuss both population-mechanism inspectability and the countervailing ease of inspecting one complete agent.

8.3. RQ3: Schedule semantics

- Compare schedules that intentionally expose different information or update timing.
- For a fixed phased semantics, report trajectory equality across admissible system orders.
- State the dependency, query-stability, randomness, resolution, and reduction conditions used for each equivalence claim.
- Present counterexamples showing how outcomes change when a required ordering or phase boundary is removed.

8.4. Secondary engineering results

- Separate data-layout gains from threading gains.
- Show scaling curves rather than a single population size.
- Identify cross-over points where ECS overhead becomes worthwhile.
- Report workloads where ECS provides little or no advantage.

9. Discussion

9.1. What the three answers jointly show

Synthesize the evidence rather than restating the architecture:

- RQ1 establishes how component signatures affect the representation and modification of overlapping and changing roles relative to an idiomatic agent-centered implementation of the same model.
- RQ2 establishes how system boundaries affect the locality, reuse, substitutability, and inspectability of population-level mechanisms.
- RQ3 establishes how dependencies and phase boundaries specify information visibility and update timing, and which declared conditions make alternative execution orders outcome-preserving.

The secondary engineering results should then report whether the same component and dependency information improves execution for the tested workloads without being presented as a fourth answer.

This synthesis should answer the agency question briefly. An agent remains a scientific unit with identity, private state, information, feasible actions, and consequences. A software object is one implementation of that unit; ECS externalizes shared transition laws without eliminating entity-specific state or scientifically meaningful autonomy.

9.2. Where the alignment breaks down

- It does not determine the scientifically correct timing semantics.
- It does not make conflicting interactions automatically parallel.
- It does not guarantee cache or GPU performance.
- It does not remove the need for validation, calibration, or sensitivity analysis.
- It may make an individual agent's complete behavior harder to inspect.
- It can be excessive for small models with a single stable agent type and strongly individualized logic.

9.3. Implications for economic ABMs

Develop examples beyond Sugarscape:

- A firm can acquire exporter, borrower, employer, or innovator capabilities without becoming a new nominal type for every combination.
- A household can participate in labor, credit, housing, and consumption systems according to its current components.
- Institutions can be modeled as systems or resources rather than necessarily as agents, forcing the modeler to state where agency is substantively intended.
- Entry, bankruptcy, learning, and institutional change can be represented as transformations of component composition.

These examples should remain implications unless they are implemented as additional case studies.

9.4. Threats to validity

- One Sugarscape model cannot establish universal architectural superiority.
- The agent-centered implementation may reflect author familiarity or framework-specific constraints.
- The chosen ECS library may conflate the abstract pattern with a particular storage implementation.
- Performance results may be hardware- and workload-specific.
- Dynamic composition can introduce semantic choices about when component changes take effect.
- The distinction between an entity, an agent, and an environmental object must be documented explicitly.

10. Conclusion

Entity Component Systems should be evaluated in ABM not only as a performance technique but as an alternative architecture for model specification. Their main scientific promise lies in representing heterogeneous agents as changing compositions of capabilities, organizing behavior as explicit population-level processes, and exposing the dependencies involved in simultaneous interactions. Cache-efficient storage and parallel execution are important consequences, but their benefits remain empirical and workload-dependent.

The next research step is to test this alignment in a larger economic model with multiple institutional roles and endogenous changes in agent capabilities.

A Complete ODD description

A.1 Purpose, entities, environment, and scale

The model studies how local movement and harvesting on a regenerating, spatially unequal resource landscape generate a distribution of wealth, and how reproduction and disease alter that distribution. Citizens are agents; sugar patches form a toroidal environment. One application of the declared schedule advances one discrete period.

A.2 State and initialization

Core citizen state comprises identity, position, proposal, vision, metabolism, sugar, age, maximum age, initial endowment, sex, and immune profile. `Infection` is optional. Setup places the requested population in distinct cells using the seeded RNG and initializes a deterministic two-hill capacity landscape unless a nonnegative capacity matrix is supplied.

The default parameters use a 50×50 grid, 400 citizens, patch capacity four, growback one, vision one to six, metabolism one to four, initial sugar five to 25, and lifespan 60 to 100 periods. The baseline replaces deaths. Reproduction and infection are disabled unless explicitly enabled.

A.3 Process overview

Each period grows patch sugar, rebuilds occupancy, executes one of the two movement schedules, processes disease, metabolizes and ages citizens, removes deaths, creates replacements or offspring as configured, advances the clock, and records aggregate outcomes. All stochastic choices use the seeded model RNG. Structural additions and removals are staged and committed between queries.

A.4 Movement and harvesting

A citizen considers its current cell and cardinal cells up to its vision on the torus. Occupied cells are ineligible except for its own current cell. It maximizes current patch sugar, minimizes distance among equal-sugar cells, and uses the RNG for remaining ties. The destination is harvested to zero and its sugar is added to citizen wealth.

Sequential movement updates occupancy and landscape sugar after each citizen. Synchronous movement freezes them for proposal, groups proposals by destination, selects one winner for each contested cell, commits winners and nonmoving losers, and rebuilds occupancy.

A.5 Disease, lifecycle, and population regeneration

Disease transmission uses cardinal post-movement neighbors. A susceptible citizen with no exact-strain immunity may acquire one neighboring strain. Infection age and sugar cost update in the same period; reaching the disease duration stores strain immunity and removes `Infection`.

Metabolism subtracts the citizen's metabolic rate and age increases by one. Nonpositive sugar causes starvation; age above maximum age causes old-age death. Removed citizens are replaced only in the replacement regime. Otherwise, when reproduction is enabled, an eligible adjacent female-male pair may reserve an empty neighboring cell and contribute half of each

parent's initial endowment to a child. The child independently inherits vision, metabolism, and maximum age from either parent and receives a sex tag.

A.6 Recorded outcomes

The logger records population, mean and median wealth, the Gini coefficient, mean age, total citizen sugar, total landscape sugar, movements, conflicts, harvested sugar, deaths by cause, replacements, births, infection prevalence, incident infections, and recoveries. Complete citizen snapshots provide deterministic state-level comparisons between implementations and schedules.

B Supplementary architecture comparison

- Full agent-centered pseudocode.
- Full ECS system table with queries, reads, writes, and structural changes.
- Dependency graph and execution phases.

C Supplementary experimental design

- Infection-role and reproduction treatments.
- Density, resource, horizon, and disease-parameter grids.
- Replicate counts and seed construction.
- Primary and secondary outcomes.

D Additional results

- Full movement-schedule comparisons.
- Reproduction and disease sensitivity.
- Longer-horizon, landscape, and initialization robustness.
- State-equivalence results for admissible execution orders.
- Complete performance profiles.

E Reproducibility

- Repository and archived release.
- Julia, Rust, Typst, and package versions.
- Hardware and operating-system description.
- Commands for tests, simulations, figures, benchmarks, and paper compilation.

Bibliography

[1] J. M. Epstein and R. L. Axtell, *Growing Artificial Societies: Social Science from the Bottom Up*. Washington, DC and Cambridge, MA: Brookings Institution Press, MIT Press, 1996. Accessed: Aug. 13, 2026. [Online]. Available: https://www.brookings.edu/books/growing-artificial-societies/

[2] S. Abar, G. K. Theodoropoulos, P. Lemarinier, and G. M. P. O'Hare, "Agent Based Modelling and Simulation Tools: A Review of the State-of-Art Software," *Computer Science Review*, vol. 24, pp. 13–33, May 2017, doi: 10.1016/j.cosrev.2017.03.001.

[3] E. ter Hoeven *et al.*, "Mesa 3: Agent-Based Modeling with Python in 2025," *Journal of Open Source Software*, vol. 10, no. 107, p. 7668, 2025, doi: 10.21105/joss.07668.

[4] S. Luke, C. Cioffi-Revilla, L. Panait, K. Sullivan, and G. Balan, "MASON: A Multiagent Simulation Environment," *SIMULATION*, vol. 81, no. 7, pp. 517–527, 2005, doi: 10.1177/0037549705058073.

[5] U. Wilensky, "NetLogo." Accessed: Aug. 12, 2026. [Online]. Available: http://ccl.northwestern.edu/netlogo/

[6] G. Datseris, A. R. Vahdati, and T. C. DuBois, "Agents.jl: A Performant and Feature-Full Agent-Based Modeling Software of Minimal Code Complexity," *SIMULATION*, vol. 100, no. 10, pp. 1019–1031, 2024, doi: 10.1177/00375497211068820.

[7] P. Richmond, R. Chisholm, P. Heywood, M. K. Chimeh, and M. Leach, "FLAME GPU 2: A Framework for Flexible and Performant Agent-Based Simulation on GPUs," *Software: Practice and Experience*, vol. 53, no. 8, pp. 1659–1680, 2023, doi: 10.1002/spe.3207.

[8] P. Redmond, J. Castello, J. M. Calderón Trilla, and L. Kuper, "Exploring the Theory and Practice of Concurrency in the Entity-Component-System Pattern," *Proceedings of the ACM on Programming Languages*, vol. 9, no. OOPSLA2, pp. 1–28, Oct. 2025, doi: 10.1145/3763050.

[9] A. Ambrosio, D. D. Vinco, F. Foglia, C. Spagnuolo, and V. Scarano, "The Impact of ECS Logic on Parallel Performance in Agent-Based Model Simulations."

[10] A. Casals and A. A. F. Brandão, "HECATE: An ECS-based Framework for Teaching and Developing Multi-Agent Systems." Accessed: Feb. 02, 2026. [Online]. Available: http://arxiv.org/abs/2509.06431